Merge Sort :

Code:

```c
#include <stdio.h>
#include <stdlib.h>

void merge(int arr[], int l, int m, int r){

    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    int L[n1], R[n2];

    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = 0;
    j = 0;
    k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
```

```
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(int arr[], int l, int r){

    if (l < r) {
        int m = l + (r - l) / 2;

        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}

int main(){

    int arr[] = {157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133};
    int arr_size = sizeof(arr) / sizeof(arr[0]);

    mergeSort(arr, 0, arr_size - 1);
    int i;
    for (i = 0; i < arr_size; i++)
        printf("%d ", arr[i]);
    printf("\n");

    return 0;
}
```

Output:

```
[keermanipamisetty@Keermanis-MacBook-Air DAA LAB % gcc mergesort.c -o mergesort
[keermanipamisetty@Keermanis-MacBook-Air DAA LAB % ./mergesort
110 111 112 117 122 123 133 141 147 149 151 157
```

Complexity Analysis of Merge Sort

- **Best Case: O(n log n)**, when the array is already sorted or nearly sorted.
- **Average Case: O(n log n)**, when the array elements are in random order.
- **Worst Case: O(n log n)**, when the array is sorted in reverse order.

Quick Sort :

Code:

```c
#include <stdio.h>

void swap(int* a, int* b);

int partition(int arr[], int low, int high) {

    int pivot = arr[high];

    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {

        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```c
int main() {
    int arr[] = {157, 110, 147, 122, 111, 149, 151, 141, 123, 112, 117, 133};
    int n = sizeof(arr) / sizeof(arr[0]);

    quickSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output:

```
[keermanipamisetty@Keermanis-MacBook-Air DAA LAB % gcc quicksort.c -o quicksort  ]
[keermanipamisetty@Keermanis-MacBook-Air DAA LAB % ./quicksort                   ]
 110 111 112 117 122 123 133 141 147 149 151 157 %
```

Complexity Analysis of Quick Sort

• **Best Case: O(n log n)**, when the pivot divides the array into two nearly equal halves at each step.

• **Average Case: O(n log n)**, when the pivot generally splits the array into reasonably balanced parts for random input.

• **Worst Case: O(n²)**, when the pivot is always the smallest or largest element (for example, when the array is already sorted or reverse sorted and a poor pivot is chosen).